

Extraction of test cases procedures from textual use cases to reduce test effort: Test Factory Experience Report

Erick Barros dos Santos*
erickbarros@great.ufc.br
Grupo de Redes de Computadores,
Engenharia de Software e Sistemas
Universidade Federal do Ceará
Fortaleza, Ceará

Lucas Simão da Costa
lucascosta@great.ufc.br
Grupo de Redes de Computadores,
Engenharia de Software e Sistemas
Universidade Federal do Ceará
Fortaleza, Ceará

Bruno Sabóia Aragão†
bruno@great.ufc.br
Grupo de Redes de Computadores,
Engenharia de Software e Sistemas
Universidade Federal do Ceará
Fortaleza, Ceará

Ismayle de Sousa Santos
ismaylesantos@great.ufc.br
Grupo de Redes de Computadores,
Engenharia de Software e Sistemas
Universidade Federal do Ceará
Fortaleza, Ceará

Rossana Maria de Castro
Andrade‡
rossana@great.ufc.br
Grupo de Redes de Computadores,
Engenharia de Software e Sistemas
Universidade Federal do Ceará
Fortaleza, Ceará

ABSTRACT

The optimization of processes in the software development life cycle has become mandatory, since companies need to reduce the costs of their projects and shorten the time to market without affecting the quality of the product. With regards to software products, activities related to the testing process play a major role in their quality, since they assure that the product complies with the elicited requirements. However, companies usually do not perform testing activities on software projects due to financial and time constraints. These costs may be even higher in testing processes that rely on manual execution of activities, which is error-prone and can take too much time. To mitigate this situation, test automation allows the reduction of the effort required. This paper presents an experience report in a Test Factory about the use of a tool that partially automates the specification of test procedures based on use cases. The tool aims to read the textual use cases from the requirement management system, extract the test case procedures and required inputs to manual editing, and then send the test cases to the test management system. This solution is then applied to software from the telecommunications industry. This paper also presents how the tool was used and the lessons learned during the whole process.

*Bolsista de Mestrado do CNPq.

†Bolsista da Fundação de Cultura e Apoio ao Ensino, Pesquisa e Extensão.

‡Bolsista do CNPq de Produtividade em Desenvolvimento Tecnológico e Extensão Inovadora (DT) 2

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBQS'19, October 28-November 1, 2019, Fortaleza, Brazil

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7282-4/19/10...\$15.00

<https://doi.org/10.1145/3364641.3364673>

CCS CONCEPTS

• **Software and its engineering** → **Software verification and validation.**

KEYWORDS

Software Testing, Test Generation, Test Factory

ACM Reference Format:

Erick Barros dos Santos, Lucas Simão da Costa, Bruno Sabóia Aragão, Ismayle de Sousa Santos, and Rossana Maria de Castro Andrade. 2019. Extraction of test cases procedures from textual use cases to reduce test effort: Test Factory Experience Report. In *XVIII Brazilian Symposium on Software Quality (SBQS'19), October 28-November 1, 2019, Fortaleza, Brazil*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3364641.3364673>

1 INTRODUÇÃO

Testes de software desempenham um papel fundamental na garantia de qualidade do software, permitindo que defeitos sejam encontrados antecipadamente no ciclo de vida do produto e, consequentemente, diminuindo custos de manutenção de software [17]. Entretanto, erros no planejamento dos testes e nas estimativas de esforço podem contribuir para o surgimento de dívidas técnicas de teste [19]. Adicionalmente, a execução de atividades de testes manuais podem ser demoradas e propensas a erros. Sendo assim, casos de teste especificados incorretamente ou com baixa cobertura dos requisitos podem levar ao retrabalho e/ou baixa detecção de erros.

Para reduzir o esforço associado às atividades de testes, as abordagens e ferramentas mais populares estão relacionadas à automação, uma vez que permitem reduzir a execução manual de tarefas [5]. Como discutido por Garousi e Mäntylä [8], a automação de testes teve como foco inicial a execução dos testes. Entretanto, como relatado em um trabalho anterior dos autores deste artigo [25], desenvolver e manter scripts de testes pode demandar muito esforço de uma equipe.

Apesar do foco em automatizar a execução dos testes, projetos na indústria também têm se dedicado à automação de outras atividades, tais como geração de casos de teste e relatórios dos testes [8]. No que se refere a pesquisa de geração de casos de teste, Anand et al. [1] listam os seguintes tópicos como os mais proeminentes da área: execução simbólica, teste baseado em modelos, teste combinatorial, teste randômico e teste baseado em busca. Apesar dos avanços no desenvolvimento desses tópicos na academia, a maior preocupação está em melhorar a efetividade e eficiência dos testes com a automação e uso de técnicas de fácil implantação prática [7].

Neste sentido, o presente artigo tem o objetivo de relatar a experiência sobre a automação das atividades de geração de procedimentos de testes em uma Fábrica de Testes [20]. Para isso, foi feita uma pesquisa na literatura por soluções que automatizassem atividades semelhantes. Em seguida, as soluções que mais se adequavam a um projeto da indústria serviram de base para a elaboração de uma ferramenta customizada. Como contribuição, serão apresentados os dados sobre o impacto do uso da ferramenta no esforço das atividades de teste e as lições aprendidas durante o estudo.

O restante deste artigo está estruturado como segue: a Seção 2 apresenta os principais trabalhos relacionados ao presente artigo; na Seção 3 são apresentadas informações sobre o ambiente da experiência, detalhando a natureza do projeto, perfil da equipe, ferramentas e processos; a Seção 4 apresenta a metodologia seguida para elaboração e aplicação da solução na indústria; a Seção 5 detalha o uso da solução elaborada e os dados coletados durante o processo; a Seção 6 sumariza as lições aprendidas obtidas durante a execução do estudo; por fim, a Seção 7 apresenta a conclusão do artigo.

2 TRABALHOS RELACIONADOS

Na literatura estão disponíveis diversos trabalhos sobre geração de procedimentos e casos de testes a partir de casos de uso. Algumas abordagens se sustentam no Processamento de Linguagem Natural (PLN) [18] para a extração de casos de teste, e outras na geração de modelos intermediários para extrair as informações necessárias [14]. Trabalhos na literatura têm realizado avaliações e relatos de experiência sobre ferramentas e abordagens de geração de testes na indústria, de forma que os mesmos encontram-se descritos a seguir.

Jorge et al. [12] propõem CLARET, uma linguagem específica de domínio que permite a criação de casos de uso usando linguagem natural estruturada e casos de teste. É apresentada uma ferramenta de suporte que permite não só especificar e verificar os casos de uso, como também convertê-los em diagramas de Sistema de Transição Rotulado para a geração dos testes. No trabalho são descritos estudos de caso na indústria no contexto de projetos de desenvolvimento ágil, de forma que engenheiros de software especificaram casos de uso utilizando a CLARET e geraram testes com uma ferramenta auxiliar. Ao final são apresentados lições aprendidas e resultados de esforço que afirmam a efetividade da solução.

Gutiérrez et al. [10] também apresentam uma abordagem de geração de casos de teste baseado em modelos, mas focando no uso de metamodelos para aumentar a generalidade do uso da solução com diferentes abordagens. Para isso, a solução utiliza os metamodelos para modelar casos de uso e elementos de testes, realizando assim

transformações entre os modelos até obter os casos de teste. O trabalho apresenta três casos de uso na indústria, sendo um deles no contexto ágil, e sumariza também as lições aprendidas. Mesmo com a introdução de modelos extras e suas respectivas transformações, foram relatados reduções de esforço com o uso das ferramentas. Entretanto, poucas informações são fornecidas sobre esforço da abordagem no contexto ágil.

Yue et al. [27] apresentam a RTCM, uma abordagem para a especificação textual de casos de teste usando elementos similares aos presentes em casos de uso. A abordagem oferece uma série de padrões para a especificação dos testes e a ferramenta aToucan4Test, cuja funcionalidade é auxiliar no processo de geração de casos de teste manuais e automáticos. Para analisar a viabilidade da solução, dois estudos de caso industriais do domínio de sistemas ciber-físicos foram utilizados com a ferramenta. A avaliação foi focada na geração de scripts de testes automatizados, em que os autores relataram redução significativa no esforço de implementação. Por fim, os autores apresentam lições aprendidas do processo.

Massollar et al. [14] apresentam uma abordagem automatizada baseada em modelos para a geração de casos de teste. A abordagem consiste em especificar os casos de uso utilizando padrões próprios, de forma que estes são convertidos em diagramas de atividades UML¹ para representar o comportamento do sistema. Os diagramas de atividade têm uma função dupla: checar se os casos de uso foram especificados corretamente e auxiliar a geração de um modelo de testes. Este último modelo é a base para a geração dos procedimentos e casos de teste, de forma que o analista de teste deve identificar e inserir manualmente os dados para que os casos de teste sejam gerados. Ao final, foi realizada uma avaliação da ferramenta com dois engenheiros de software e um grupo de estudantes. Foram apresentados dados sobre tempo de especificação e verificação de modelos, mas pouca ênfase foi oferecida para a geração dos testes.

O presente trabalho teve como objetivo focar na geração automática de procedimentos de testes a partir de casos de uso textuais. Diferentes de outros trabalhos na literatura [18, 23], optou-se por não se utilizar de PLN na solução elaborada. Para fazer a análise e extração dos casos de uso, decidiu-se por utilizar estruturas bem definidas nos casos de uso, mas sem restringir a especificação dos passos com uma gramática de sintaxe bem definida. Neste sentido, um dos trabalhos mais relacionados à solução elaborada é o de Massollar et al. [14], de forma que foram utilizados padrões de casos de uso e sequências de atividades semelhantes, bem como também foi realizada uma avaliação na indústria. As diferenças com relação a esse trabalho é de que não houve foco na etapa de checagem de requisitos e procurou-se evitar interação com modelos adicionais.

Os pontos apresentados anteriormente permitiram que os analistas de testes focassem na geração de procedimentos de testes, reduzindo assim o esforço extra associado com a manipulação de modelos. Como diferencial em relação aos trabalhos relacionados, o presente artigo oferece lições aprendidas e discute resultados de métricas de esforço durante a geração automática de procedimentos de teste na indústria, focando no contexto de uma Fábrica de Testes trabalhando com um projeto ágil.

¹<https://www.uml.org/>

3 AMBIENTE DA EXPERIÊNCIA

Esta seção tem como objetivo apresentar uma visão geral do ambiente em que foi realizada a experiência. A Subseção 3.1 provê algumas informações sobre o projeto de requisitos e testes. A Subseção 3.2 sumariza o perfil da equipe da indústria que trabalhou no referido projeto. A Subseção 3.3 descreve as ferramentas e padrões adotados no projeto. A Subseção 3.4 e 3.5 detalham, respectivamente, os processos de requisitos e testes utilizados pela equipe da Fábrica de Testes. Esses processos foram elaborados baseados nas experiências anteriores[2, 25] da Fábrica de Testes do GREAt² com projetos de testes.

3.1 Projeto de Requisitos e Testes

A experiência foi realizada em um projeto de Pesquisa, Desenvolvimento e Inovação, de forma que as atividades realizadas na organização são relacionadas à elicitação de requisitos e testes no Software A³. Este software tem como objetivo o gerenciamento de equipamentos pertencentes a uma rede óptica passiva. O projeto tem um caráter distribuído, pois o cliente, a equipe de desenvolvimento e a equipe de requisitos/testes pertenciam a organizações diferentes.

A equipe de requisitos/testes do projeto do Software A possui um processo baseado no *framework* SCRUM [21] com *sprints* de duração de um mês. Até a data da execução da experiência, haviam sido realizadas um total de nove *sprints* e com previsão de realização de mais sete. As *sprints* definidas para o projeto possuíam tempo fixo, de forma que se objetivava concluir o escopo definido para ela. Porém, *sprints* futuras poderiam ser alteradas para incluir mais atividades relacionadas a elicitação de requisitos, testes e retestes de funcionalidades e defeitos.

3.2 Perfil da Equipe

A equipe da Fábrica de Testes envolvida no projeto do Software A era composta por: um gerente, dois analistas de requisitos/testes, um estagiário (testador) e um pesquisador. Dentre os integrantes, apenas um dos analistas e o estagiário realizavam a execução dos casos de teste. O analista possuía uma experiência de dez meses com elicitação de requisitos, e um ano e dois meses com análise e execução de testes. O estagiário possuía um ano de experiência tanto em elicitação de requisitos, quanto em análise e execução de testes. O analista e estagiário tinham, respectivamente, oito e dez meses de experiência em atividades de requisitos e testes no Software A.

Os analistas do projeto realizavam tanto as atividades de elicitação de requisitos, na forma de casos de uso, quanto de testes. Os casos de uso elaborados servem de base para especificação dos testes. Além dos testes baseados em casos de uso, a equipe também realizava testes exploratórios durante a execução das *sprints*. Esse elevado conhecimento sobre os requisitos do Software A permitia aos analistas gerar uma documentação de testes mais sucinta, confeccionando assim mais agilidade durante o processo. Portanto, a execução dos testes é suportada pelos documentos gerados, mas é baseada na experiência dos analistas. Principalmente em um projeto com

muitas mudanças de requisitos e entregas com datas fixas, mesmo a geração de uma documentação sucinta de testes pode ser onerosa.

3.3 Tecnologias e Padrões Utilizados nos Processos Internos

Para guiar as atividades dos processos de elicitação de requisitos e testes (ver Seções 3.4 e 3.5), a equipe descrita na Seção 3.2 fazia uso das seguintes ferramentas: JIRA⁴, para a gestão de tarefas e casos de uso; Confluence⁵, para o armazenamento de regras de negócio e documentações em geral; TestLink⁶, para a gestão dos planos de teste e execução dos casos de teste; navegadores Google Chrome⁷ e Firefox⁸, para a execução dos casos de teste.

Desde o princípio do projeto do Software A, optou-se por realizar a especificação de casos de uso utilizando *templates* bem definidos, mesmo que não houvesse um direcionamento bem definido sobre a geração automática de procedimentos ou casos de teste a partir dos requisitos. Adicionalmente aos *templates*, também foram utilizados símbolos especiais que permitissem uma fácil identificação de elementos específicos nos passos dos casos de uso. A Figura 1 ilustra um exemplo de caso de uso utilizado no projeto, em que o Fluxo Básico inicia na linha 14 seguindo o rótulo [Fluxo Básico]. Na linha 16 da mesma figura, os campos de entrada de texto são identificados com aspas duplas. Na linha 18, os elementos visuais clicáveis do sistema devem aparecer entre os símbolos <>.

```

1  Título: UC001.01 - Editar Cadastro de Usuário
2
3  [Objetivo]
4  - Permitir usuário a realizar edição de usuários
5
6  [Mockup]
7  - TL001 - Editar Usuário
8
9  [Pré-condição]
10 - Estar logado no sistema com usuário que tenha permissões para
11 realizar edição de usuários do sistema
12 - Estar com a tela 'TL002 - Editar Usuário' aberta
13
14 [Fluxo Básico]
15 1. O usuário pesquisa por um usuário
16 2. O usuário preenche os campos "Nome", "CPF", "E-mail", "Senha",
17 "Confirmação de Senha" e "Tipo de Usuário" [RN-01]
18 3. O usuário clica em <Salvar> [FE-01]
19 4. O sistema apresenta a mensagem: Usuário editado com sucesso.
20
21 [Fluxos de Exceção]
22
23 [FE-01] - E-mail já cadastrado
24 1. A partir do passo 3. do [Fluxo Básico]
25 2. O sistema verifica que o novo e-mail já está cadastrado no sistema e
26 apresenta a seguinte mensagem: E-mail já cadastrado.
27 3. Retorna para o passo 3. do [Fluxo Básico]
28
29 [Critérios de Aceitação]
30 - [Fluxo Básico]
31 - [FA-01]
32 - [FE-01]
```

Figura 1: Exemplo de padrões seguidos para a especificação de casos de uso.

⁴<https://www.atlassian.com/software/jira>

⁵<https://www.atlassian.com/software/confluence>

⁶<http://www.testlink.org/>

⁷<https://www.google.com/chrome/>

⁸<https://www.mozilla.org/>

²<http://www.great.ufc.br>

³O nome do sistema foi omitido por conta de contratos de confidencialidade

3.4 Processo de Requisitos

A fim de sistematizar as atividades de engenharia de requisitos no projeto, foi definido um processo de requisitos. Esse processo é composto pelas atividades: Elicitação, Análise, Especificação e Validação. Essas etapas são consideradas essenciais para a engenharia de requisitos em projeto de desenvolvimento de software [26].

Ao longo do projeto, as atividades relacionadas aos requisitos eram executadas de tal forma que suas saídas serviam como insumo para as tarefas do *backlog* da *sprint* do time de desenvolvimento e testes. Mesmo se tratando de um projeto com características ágeis, o cliente solicitou que fosse elaborada uma documentação mais detalhada para o Software A por conta de suas funcionalidades complexas. Por conta disso, os requisitos foram especificados na forma de casos de uso. Abaixo, de forma resumida, seguem as atividades realizadas em cada etapa.

- (1) **Elicitação:** Busca identificar os requisitos do sistema através da consulta com *stakeholders*. No processo em questão, algumas técnicas são utilizadas: Entrevistas com os *stakeholders*; Criação de cenários de uso e Prototipação utilizando a ferramenta Balsamiq⁹.
- (2) **Análise:** Verifica, por exemplo, consistência, completude e viabilidade dos requisitos inicialmente elicitados. Nessa etapa, os requisitos são também priorizados a fim de identificar quais requisitos darão maior retorno de investimento para o cliente e usuários finais.
- (3) **Documentação:** Os requisitos são documentados em formato de casos de uso e comunicados a toda equipe. Vale ressaltar que as regras de negócio do sistema também são documentadas.
- (4) **Validação:** Esta etapa tem o objetivo de garantir que os requisitos estão com descrição aceitável e estão aptos a serem disponibilizados para desenvolvimento. No projeto deste relato, essa etapa envolvia também a criação e validação dos protótipos de alta fidelidade.

3.5 Processo de Testes

O processo de teste fornece um *feedback* real do comportamento do software que está sendo executado [3] e quando as atividades são organizadas em um processo, elas podem ser comunicadas, seguidas, observadas e melhoradas [15].

Os processos podem variar de acordo com as necessidades de cada organização. Existem, porém, aqueles que são genéricos e podem ser utilizados nas organizações com algumas adaptações, se necessário [15] [11]. O processo utilizado na Fábrica de Testes do GREat [4] é baseado no MPS.BR [16] e é composto de três fases: (i) Planejamento; (ii) Elaboração; e (iii) Execução. No contexto do projeto do relato, as atividades de testes recebem como entrada os requisitos especificados no processo descrito na Seção 3.4, de forma que a especificação e execução dos testes é realizada antes da liberação do sistema. Uma descrição resumida das principais atividades desse processo é apresentada a seguir.

- (1) **Planejamento:** Consiste em verificar o objetivo dos testes e realizar as ações necessárias para transformar a estratégia de teste em um plano operacional.

- (2) **Especificação:** Tem o propósito de elaborar os testes de forma que atenda às demandas do plano de teste. Inclui também a elaboração dos scripts de testes automatizados, se pertinente.
- (3) **Execução:** Consiste em realizar o teste e registrar a execução e os resultados. Nessa fase também são analisados os incidentes de testes. Além disso, nessa fase é gerado o relatório de teste a ser enviado ao cliente e a documentação das lições aprendidas.

Vale ressaltar que durante todo o processo é realizado um monitoramento e controle das atividades executadas a fim de, se necessário, fazer as correções dos desvios identificados.

4 METODOLOGIA DO ESTUDO

Para guiar a execução da experiência, foi utilizada uma metodologia com passos baseados no modelo de transferência de tecnologia proposto por Gorscheck et al. [9]. A metodologia utilizada no relato tem um total de quatro passos, em que a principal diferença em relação ao modelo de transferência original é o agrupamento e simplificação das fases de validação em uma fase única. Os detalhes desses passos são descritos a seguir.

Passo 1 - Identificar potenciais áreas de melhoria baseadas em necessidades da indústria: Neste passo, foram realizadas observações na execução das atividades relacionadas aos testes no Software A. Para auxiliar no levantamento de informações, o pesquisador discutiu possíveis demandas junto ao Gerente e Líder Técnico do projeto. Dessa forma, foi possível identificar pontos de melhoria na execução de processos de especificação de testes. Em seguida, os analistas de requisitos/testes foram consultados para obter mais detalhes sobre como as atividades de testes poderiam ser realizadas para demandar menos esforço¹⁰.

A partir dos comentários coletados com toda equipe, foi identificada a necessidade de uma ferramenta de automação que auxiliasse na execução dos processos de análise e especificação de procedimentos de teste. Neste sentido, o presente estudo foi realizado para identificar a viabilidade de inserir uma ferramenta semi-automatizada para reduzir o esforço dessas atividades.

Passo 2 - Elaborar solução: Após o passo anterior, deu-se início à fase de elaboração de uma solução. O objetivo era utilizar conceitos e práticas de soluções existentes que melhor se ajustassem ao atual projeto do Software A. Para isso, foi feito um breve levantamento de algumas soluções presentes na literatura que pudessem servir de base para a elaboração de uma solução customizada. Como apresentado na Seção 2, muitas abordagens são suportadas por modelos adicionais para aumentar a efetividade dos testes gerados. Entretanto, foi escolhido evitar abordagens dependentes de modelos, pois o objetivo era uma solução de fácil implantação que não exigisse a manipulação de um formalismo adicional ou que pudesse de alguma forma afetar a execução da *sprint*. Como os casos de uso do Software A foram especificados no idioma português, também optou-se por não utilizar abordagens baseadas em PLN, uma vez que as soluções encontradas são projetadas para o uso com o idioma inglês. Adicionalmente, não se tinha a intenção de utilizar uma linguagem estruturada muito rígida para não impactar na especificação dos casos de uso.

⁹<https://balsamiq.com/wireframes/>

¹⁰O esforço será calculado em homem-hora

Além das limitações mencionadas anteriormente, a equipe da Fábrica de Testes já havia definido uma *template* a ser utilizada na elicitação de requisitos, de forma que a equipe solicitou que não houvesse alteração dos mesmos. Dentre as outras soluções na literatura que propõem geração de testes, a Specmate [6] é uma das ferramentas atuais com mais funcionalidades. Apesar da mesma ainda depender de modelos adicionais para a geração dos testes, seu processo de especificação de procedimentos e inserção de dados de teste era direto e não dependia de modelos adicionais. Entretanto, a mesma não oferecia o suporte necessário para a extração de informações dos casos de uso a partir do JIRA e nem o envio de testes para a ferramenta TestLink.

No que diz respeito à geração de testes, Somé e Cheng [24] propõe uma abordagem para a geração de cenários de teste a partir de casos de uso textuais especificados em linguagem restrita. Nessa abordagem, um caso de uso é convertido em uma de máquina de estados que permite gerar caminhos entre os fluxos. No fim do processo, o analista de teste deve manualmente identificar as variáveis de interesse e inserir os dados de teste. Dado a praticidade dessa abordagem, optou-se por utilizar uma sequência de passos similar para o desenvolvimento da solução deste relato.

Neste sentido, foi desenvolvida uma ferramenta para a automação do projeto com alguns passos semelhantes aos descritos por Massollar et al. [14] e Somé e Cheng [24]. Ao contrário da abordagem de Massollar et al. [14], optou-se por não focar na checagem automática dos requisitos. As principais diferenças em relação ao trabalho de Somé e Cheng [24] encontram-se no uso de critérios de cobertura mais simples e nos padrões dos casos de uso. O *template* utilizado no projeto possibilitou a identificação automática de variáveis de entrada, referências a páginas, elementos cliváveis e variáveis de entrada nos passos dos casos de uso. No que diz respeito à interface que permitia manipular os procedimentos, optou-se por utilizar interações baseadas na ferramenta Specmate.

Dadas as considerações mencionadas anteriormente, optou-se pelo desenvolvimento de uma ferramenta que realizasse a automação parcial do processo de especificação de procedimentos de testes. Dessa forma, os analistas poderiam ter mais controle sobre a execução da especificação. Adicionalmente, também era possível inserir dados de entrada para criar casos de teste. Durante o processo de desenvolvimento, todos os integrantes foram consultados durante o levantamento das funcionalidades da ferramenta.

A ferramenta foi desenvolvida como um *Progressive Web App* utilizando o *framework* Ionic¹¹. As funcionalidades da mesma incluem: leitura de casos de uso estruturados na ferramenta JIRA; geração de procedimentos de teste a partir de fluxos dos casos de uso; alteração dos procedimentos de teste extraídos; inserção de dados para criar casos de teste; e, por fim, envio de casos de teste gerados para a ferramenta TestLink. Essas funcionalidades encontram-se detalhadas a seguir.

- (1) **Integração com o JIRA:** O usuário pode inserir o identificador de uma tarefa de caso de uso no sistema JIRA. Em seguida, são utilizadas expressões regulares para extrair as informações dos casos de uso textuais, processando os seguintes elementos: objetivo, pré-condições, fluxos, passos, referências a outros fluxos e entradas de dados nos passos.

Para o correto funcionamento dessa operação, os casos de uso precisam estar estritamente especificados de acordo com os padrões apresentados na Subseção 3.3.

- (2) **Geração de Procedimentos de Teste:** A partir das informações extraídas dos casos de uso textuais, são gerados os procedimentos. Para isso, primeiro são criados cenários de teste, em que cada cenário é composto por sequências de fluxos a serem visitados nos casos de uso. Na geração de cenários é utilizada uma abordagem semelhante à apresentada por Somé e Cheng [24], em que o caso de uso é convertido em uma máquina de estados. Para essa funcionalidade, foi implementado um protótipo de algoritmo que gera sequências de estados iniciando no 'Fluxo Básico' e visitando todos os fluxos alternativos/exceção que partem dele. Em seguida, iniciando de cada um dos fluxos alternativo/exceção, todos os seus respectivos passos são analisados e os caminhos para outros fluxos são visitados. Por fim, as sequências de fluxos dos cenários são utilizadas para compilar os passos e variáveis de entrada que irão compor os procedimentos de teste. O critério de cobertura, apesar de simples, permite gerar cenários que passam por todos os fluxos e todas as transições. Entretanto, não é realizada uma busca em profundidade por todos os caminhos da máquina de estados, o que impede a geração de alguns cenários. A intenção desta funcionalidade é gerar procedimentos similares aos que eram gerados manualmente nos projetos e que possam auxiliar os analistas de teste do projeto na execução, gerando assim uma documentação mais sucinta.
- (3) **Gerenciamento dos Procedimentos de Teste:** A ferramenta desenvolvida permite adicionar, editar e excluir um procedimento de teste, além de permitir gerenciar os passos dentro de um procedimento. A ferramenta também extrai e exibe ao usuário as entradas listadas nos passos de cada procedimento. Essas entradas podem ser adicionadas, editadas ou removidas na edição dos passos, de forma que são armazenadas para serem reprocessados e aparecerem na lista de entradas do procedimento. Porém, não é possível extrair automaticamente referências às telas/*mockups* e atores do caso de uso.
- (4) **Inserção de Dados de Teste:** Após a geração dos procedimentos de teste, o analista de testes tem a possibilidade de inserir os dados de teste identificados e com isso, gerar instâncias de casos de teste com diferentes entradas. Isto irá compor o principal artefato de saída da ferramenta: uma suíte de casos de teste.
- (5) **Integração com o TestLink:** Após a geração dos procedimentos de teste e inserção dos dados para a geração de casos de teste, a ferramenta envia a suíte de testes gerada para o TestLink. Para que isso seja possível o usuário deve configurar o projeto no qual será criada uma suíte com os casos de teste gerados.

Passo 3 - Realizar avaliações: Para realizar uma avaliação inicial da ferramenta, um dos pesquisadores e um dos analistas executaram uma sequência de testes na ferramenta. Esses testes foram utilizados em casos de uso reais de *sprints* passadas do Software A, de forma que foi possível analisar o grau de adequação da

¹¹<https://ionicframework.com/>

Tabela 1: Métricas utilizadas para avaliação.

Grupo	Métrica	Cálculo	Interpretação
Esforço de Teste	Eficiência da Especificação de testes	Número de casos de teste especificados / Tempo total	Quanto maior, melhor
Cobertura de Teste	Requisitos de Cobertura	Número de fluxos dos casos de uso cobertos / Número total de fluxos dos casos de uso	Quanto maior, melhor
Cobertura de Teste	Casos de teste por requisito	Número total	N/A
Economia de Teste	Variância de Esforço	$[(\text{Esforço real} - \text{Esforço estimado}) / \text{Esforço estimado}] * 100$	Do resultado da fórmula é possível concluir: (i) Se a Variância de Esforço é positiva, isto significa que foi necessário tempo extra (esforço) para completar o trabalho planejado. (ii) Se a Variância de Esforço é negativa, isto significa que foi necessário menos tempo (esforço) para completar o trabalho planejado. (iii) Se a Variância de Esforço é zero, isto significa que foi necessário o esforço estimado.
N/A	Número Total	Número total de casos de teste	N/A

ferramenta aos padrões de casos de uso utilizados pela equipe. Também foi possível identificar alguns defeitos na implementação da ferramenta, sendo estes corrigidos antes do uso final da ferramenta.

Para validar a solução proposta em um uso real, foi realizado um estudo piloto em uma *sprint* do projeto do Software A que teve início no dia 07/06/2019 e planejada para finalizar no dia 04/07/2019. Todo o estudo foi conduzido por um dos pesquisadores, auxiliado pelo gerente e líder técnico do projeto. Objetivando analisar o impacto da ferramenta no trabalho da equipe, foram coletadas métricas relacionadas ao número de casos de testes, cobertura de requisitos, esforço e variância. Essas métricas faziam parte do processo da Fábrica de Testes e são baseadas em artigos disponíveis na literatura branca [22] e acadêmica [13]. A Tabela 1 sumariza essas métricas e suas respectivas fórmulas de cálculo.

Durante a execução da validação, foram feitas observações mais gerais e coleta de comentários dos analistas que serviram de insumos para as lições aprendidas presentes na Seção 6.

Passo 4 - Implantar solução: Após as avaliações, a última atividade consiste em realizar a implantação da solução. A execução deste passo está sujeito aos resultados obtidos durante a execução do estudo e ao nível de aceitação por parte da equipe de testes. Sendo assim, a solução pode sofrer alterações em suas funcionalidades para que esteja apta a entrar nos processos da Fábrica de Testes.

5 EXECUÇÃO DO ESTUDO

Nesta seção é detalhado o processo de execução do estudo após a implementação da solução como descrito na Seção 4. Na Subseção 5.1 são descritos os passos executados pela equipe de requisitos/testes durante a geração de procedimentos de testes para o Software A. Por fim, na Subseção 5.2 são sumarizados e discutidos os resultados das métricas de esforço de testes que foram coletadas.

5.1 Utilização da ferramenta de testes

Com o objetivo de auxiliar a automação parcial das atividades de especificação de testes no Software A, foi criada uma ferramenta,

cujas funções foram previamente detalhadas na Subseção 4. As atividades executadas durante o uso da ferramenta encontram-se ilustrados nas Figuras 2 e 3 e são detalhadas a seguir.

- (1) **Selecionar casos de uso da *sprint* no JIRA:** Esta atividade recebe como entrada a lista de casos de uso selecionados do *backlog* para a *sprint* atual. Neste ponto, o analista de testes deve selecionar os casos de uso para a geração automática dos procedimentos de testes.
- (2) **Verificar conformidade dos casos de uso com os padrões da organização:** Nesta atividade, o analista de testes utiliza os casos de uso da atividade anterior e verifica a conformidade deles com os padrões da organização. O foco dessa atividade é analisar se os casos de uso seguem os padrões necessários para o uso da ferramenta automatizada. As inconformidades identificadas devem ser registradas na tarefa do JIRA.
- (3) **Atualização de casos de uso:** Neste subprocesso, os casos de uso que não seguem o padrão para o uso da ferramenta devem ser atualizados pelo analista de requisitos. Dessa forma, essa atividade recebe como entrada a lista de casos de uso que não seguem o padrão. Ao final, deve-se produzir casos de uso atualizados conforme os padrões estabelecidos.
- (4) **Realizar especificação automática dos procedimentos de teste:** Esta atividade contempla o uso da ferramenta para realizar a geração automática dos procedimentos de teste. Para isso, o analista de testes deve utilizar os casos de uso verificados. Ao final do processo deve ser gerada um conjunto de procedimentos de teste.
- (5) **Analisar procedimentos de testes gerados:** Após a geração dos procedimentos, o analista de testes deve fazer uma análise dos passos gerados. Esta atividade é necessária para garantir que os passos e saídas foram corretamente extraídos a partir dos casos de uso. Caso sejam encontradas erros nos procedimentos, os mesmos devem ser registradas no sistema JIRA.

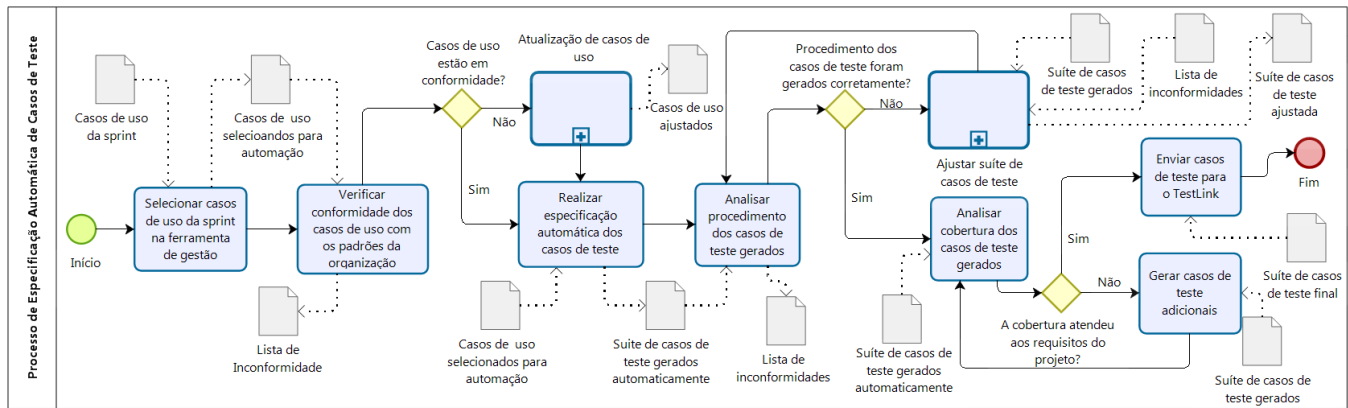


Figura 2: Processo de Especificação Automática de Procedimentos de Teste.

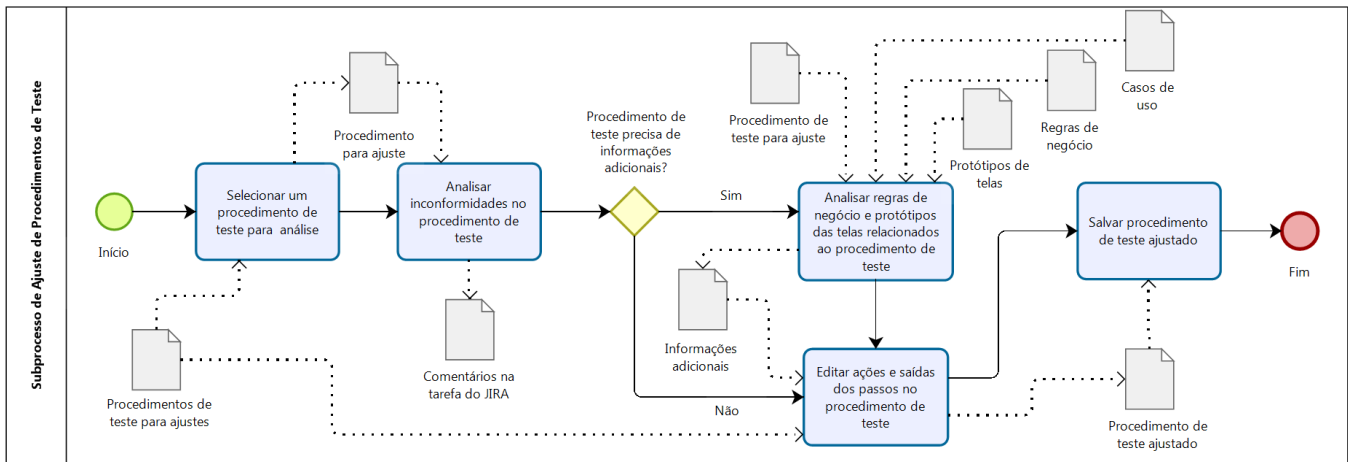


Figura 3: Subprocesso de Ajuste de Procedimento de Teste.

- (6) **Ajustar conjunto de procedimentos de teste:** Este subprocesso consiste no ajuste do conjunto de procedimentos de testes caso haja inconformidades, de forma que o analista de testes é o responsável por sua realização. O subprocesso deve receber a lista dos procedimentos de teste para ajustes e produz como saída o conjunto ajustado.
- (7) **Gerar casos de teste:** Após a análise dos procedimentos e devidos ajustes necessários, o analista de testes deve inserir dados de teste nos procedimentos para gerar os casos de teste. A atividade deve ser repetida até que se obtenha a cobertura desejada. A atividade recebe como entrada um conjunto de procedimentos de teste, e deve produzir como saída uma suite de casos de teste.
- (8) **Enviar casos de teste para o TestLink:** A última atividade do processo consiste em enviar os casos de teste para o TestLink. A atividade recebe como entrada a suite de casos de teste e deve enviar os mesmos automaticamente para a ferramenta TestLink. Nos cenários em que casos de uso são alterados durante a execução da *sprint*, é preciso gerar uma nova suite de testes que deve ser manualmente comparada e

mesclada com a versão anterior da suite, podendo ser necessário editar ou remover casos de teste dessa suite.

Após o passo (5) do processo de geração de procedimentos, ajustes manuais podem ser necessários para facilitar a execução dos casos de teste. Isso é necessário uma vez que passos possuem referências às imagens com protótipos de telas, sendo necessário a análise e extração manual das variáveis de entradas para os casos de teste. A Figura 2 ilustra as sub-atividades do passo (7) para os ajustes nos procedimentos de testes e os detalhes dessas atividades são descritos a seguir.

- (1) **Selecionar um procedimento de teste para análise:** Esta atividade recebe como entrada a lista de procedimentos de teste para ajustes. O analista de testes deve selecionar um procedimento da lista para realizar os ajustes. A saída da atividade é o procedimento de teste selecionado.
- (2) **Analisar inconformidades no procedimento de teste:** Esta atividade recebe como entrada o procedimento de teste selecionado para análise. Neste ponto, o analista de testes deve analisar as possíveis inconformidades no procedimento

gerado automaticamente. Caso haja inconformidades, o analista deve adicionar comentários na tarefa de análise e especificação de testes no sistema JIRA.

- (3) **Analisar regras de negócio e protótipos das telas relacionadas aos procedimentos de teste:** Caso a atividade anterior determine a presença de inconformidades que precisem de informações adicionais, o analista de teste deve analisar a documentação do sistema para identificar as melhorias. Para isso, essa atividade deve receber como entrada os seguintes artefatos: casos de uso, regras de negócio, protótipos de telas e procedimento de teste selecionado para ajuste. Ao final, devem ser geradas as informações adicionais para as edições dos procedimentos.
- (4) **Editar ações e saídas dos passos nos procedimentos de teste:** Nesta atividade, o analista de testes deve realizar os ajustes necessários nas ações e saídas dos procedimentos para a remoção de inconformidades. A atividade deve produzir como saída o procedimento de teste ajustado.
- (5) **Salvar procedimento de teste ajustado:** A última atividade do subprocesso compreende em salvar o procedimento de teste ajustado para que seja possível reprocessar novas variáveis de entrada. Esta atividade é realizada pelo analista de testes e recebe como entrada o procedimento ajustado.

5.2 Coleta e Discussão das Métricas

A *sprint* utilizada neste relato teve um planejamento inicial de sete casos de uso, porém, outros dois foram adicionados ao *backlog* durante a *sprint*. Desses nove casos de uso, três precisaram ser especificados dentro da *sprint*. Para a execução do estudo, foram selecionados sete casos de uso para utilizar na ferramenta, de forma que estes totalizavam 41 fluxos e 186 passos. Apesar da pequena quantidade de casos de uso na *sprint*, os mesmos representam funcionalidades complexas do Software A. O uso da ferramenta em uma *sprint* com poucos casos de uso também foi importante para analisar o impacto da ferramenta, pois a mesma não poderia dificultar a execução da *sprint*. A Tabela 2 sumariza os dados quantitativos desses requisitos.

O analista de testes especificou, para cada um dos sete casos de uso, primeiro uma suíte de casos teste manual e depois uma suíte de teste utilizando a ferramenta conforme o seu tempo disponível durante a *sprint*. Dessa forma, foram obtidos os resultados mostrados na Tabela 4, que exhibe as colunas *Tempo Fluxos*, representando o tempo para a geração dos cenários de teste e *Tempo Total*, representando o tempo total de criação dos casos de teste, considerando os cenários não explicitados no caso de uso. Para os casos de uso que possuem *Tempo Fluxos* igual ao *Tempo Total*, pode-se interpretar que não foram identificados cenários de teste além dos explicitados nos fluxos do caso de uso.

Analisando os dados das Tabelas 3 e 4, é possível identificar indícios de uma redução de esforço na geração de testes com o uso da ferramenta, principalmente para casos de uso de grande número de passos. Comparando os tempos totais de geração dos testes dos casos de uso da *sprint*, o tempo semi-automatizado levou 34,64% do tempo manual para ser concluído. Essa redução de tempo era esperada, mas não é possível concluir o seu real ganho por conta dos seguintes fatores: (i) a ferramenta foi utilizada com os casos de uso

de uma *sprint*; (ii) os testes com a ferramenta foram especificados todos depois do processo manual. Isso ocorreu dessa forma para que não houvesse muita interferência no trabalho realizado no projeto. Porém, analisando os dados e observando o *feedback* do analista e do testador, acredita-se que a solução utilizada no estudo propiciou um ganho de tempo e auxiliou a ter uma visão geral dos casos de teste.

No que diz respeito à quantidade de casos de teste gerados durante o processo, o uso da ferramenta proporcionou gerar três casos de teste a mais do que o processo manual. A ferramenta gerou um número maior de casos de teste porque quando um fluxo realiza mais de uma chamada para outro fluxo, a ferramenta gera um cenário de teste para cada chamada, cenários tais que podem passar despercebidos quando elaborados manualmente. Ainda assim, é provável que os analistas não tenham gerado mais casos de teste porque estavam mais habituados com a geração de pouca documentação e execução de testes exploratórios. Por fim, a complexidade dos casos de uso é um fator que pode impactar na geração dos testes. Como a ferramenta ainda necessitava de intervenção manual, um elevado número de fluxos alternativos/exceção demandaria dos analistas mais esforço de análise e especificação para gerar os procedimentos e dados de teste.

6 LIÇÕES APRENDIDAS

Como explicado na Seção 4, o estudo executado teve atividades relacionadas à especificação de procedimentos. Na execução dessa *sprint* foi possível obter lições aprendidas sobre a utilização da solução elaborada durante o projeto, de forma que elas são baseadas nas observações do pesquisador, opinião dos integrantes da equipe de requisitos/testes e análise das métricas coletadas. Essas lições representam alguns dos desafios obtidos com o uso da solução, de forma que as ações tomadas durante a *sprint* foram integradas ao processo de uso da ferramenta. A seguir encontram-se listadas as principais lições aprendidas durante o processo.

LA-1: A eficiência de uma ferramenta de geração de casos de teste a partir de casos de uso está fortemente relacionada ao seguimento do padrão de escrita. Modelagem de casos de uso é uma tarefa que demanda alto grau de instrução, comunicação e conhecimento do produto de software. Tratando-se de uma atividade manual, é necessário criar determinados documentos textuais suscetíveis a erros no padrão de escrita. Esses erros variam de problemas na grafia, plural ou espaços em branco nos caracteres de marcação (Como a escrita de "[FA - 01]" ao invés de "[FA-01]"), tais erros geraram falhas na ferramenta e precisaram ser corrigidos, implicando em tempo adicional no processo de geração semi-automatizado. Portanto, uma ação tomada foi a de realizar uma inspeção detalhada para determinar se os casos de uso seguem o *template* do projeto, realizando assim quaisquer correções necessárias. Utilizar um caso de uso no padrão correto como exemplo ajudou os analistas a identificarem as inconsistências mais rapidamente e, com isso, habilitando o uso da ferramenta. Para equipes que trabalham com documentação pouco detalhada, a ferramenta pode não gerar bons resultados, porém, é possível abordar em trabalhos futuros, scripts mais específicos para diferentes tipos de projetos e documentação.

Tabela 2: Casos de Uso utilizados na avaliação.

Identificador	Nº de Fluxos	Nº de Passos	Descrição
UC01	6	23	Caso de Uso de visualização de dados.
UC02	3	18	Caso de Uso de edição de status de uma entidade X
UC03	4	15	Caso de Uso de listagem de uma entidade X
UC04	5	23	Caso de Uso de movimentação de um ícone na tela
UC05	8	41	Caso de Uso de instanciação de uma entidade X
UC06	7	31	Caso de Uso de exclusão de uma entidade X
UC07	8	35	Caso de Uso de gerenciamento de uma entidade Y
Total	41	186	–

Tabela 3: Resultados das Métricas para os Casos de Uso da *Sprint*.

Métrica	UC	Resultados - Manual	Resultados - Ferramenta
Eficiência da Especificação de testes	–	0.395 ucs/min	1.219 ucs/min
Requisitos de Cobertura	–	100%	100%
Casos de teste por requisito	UC01	6	7
	UC02	3	3
	UC03	4	5
	UC04	5	5
	UC05	11	11
	UC06	7	7
	UC07	8	9
Variância de Esforço	–	-28,204	-75,129
Número Total de Casos de Teste	–	44	47

Tabela 4: Resultados de Tempos para os Casos de Uso da *Sprint*.

ID	Tempo Estimado	Nº Caso de Teste (Manual)	Tempo Fluxos (Manual)	Tempo Total (Manual)	Nº Caso de teste (Ferramenta)	Tempo Fluxos (Ferramenta)	Tempo Total (Ferramenta)
UC01	24m	6	11m 36s	11m 36s	7	2m 41s	4m 57s
UC02	15m	3	13m 01s	13m 01s	3	2m	2m
UC03	18m	4	9m 41s	9m 41s	5	20s	3m 28s
UC04	21m	5	10m 20s	12m 20s	5	5m 27s	6m 42s
UC05	30m	11	28m 46s	28m 46s	11	5m 23s	10m 23s
UC06	21m	7	8m 46s	8m 46s	7	5m 55s	7m 11s
UC07	24m 10s	8	23m	27m 7s	9	3m 52s	3m 52s
Total	2h 35m	44	1h 45m 10s	1h 51m 17s	47	25m 38s	38m 33s

LA-2: A implantação de uma ferramenta de automação pode não compensar a redução de esforço. Ao longo do projeto do Software A, o número de fluxos e passos foi utilizado para estimar o tempo necessário de geração dos testes. Em alguns casos nota-se a imprecisão deste cálculo, uma vez que, como ilustrado no UC06, um caso de uso com grande número de fluxos pode ser facilmente modelado manualmente ou de forma automatizada. Para esse caso, a ferramenta permitiu uma redução pouco significativa de ganho de eficiência, logo, o esforço de adaptar os casos de uso de um projeto para o *template* apresentado pode não compensar, principalmente se os casos de uso forem muito simples. Nestes cenários, a implantação de uma ferramenta semi-automatizada pode demandar muito trabalho manual para ajustar os casos de uso para um *template* e os procedimentos gerados, o que pode impactar nas

atividades do projeto. Entretanto, para a maioria dos casos de uso do Software A, houve indícios de que a redução do tempo para a especificação dos testes compensou a implantação da ferramenta no contexto da Fábrica de Testes.

LA-3: Só os casos de uso textuais não expressam todas as informações necessárias para uma boa cobertura de testes. Em alguns casos de uso do Software A, não foi possível obter automaticamente todas as informações necessárias para gerar mais casos de teste a partir da documentação de casos de uso. A razão disso é porque as regras de negócio eram expressas em linguagem natural não estruturada e os protótipos de tela eram imagens. Apesar de impedir a extração de algumas variáveis de entrada para os procedimentos, os padrões de casos de uso utilizados conferiam liberdade de especificação para os analistas. Durante o uso

da ferramenta, o analista de testes precisou continuar consultando os demais documentos durante o processo de análise e, com isso, garantir a cobertura desejada.

LA-4: A integração de uma solução com ferramentas específicas do processo é um fator importante para o ganho de eficiência. Durante a execução de um processo de requisitos/testes, os analistas podem precisar interagir com diferentes ferramentas de suporte para facilitar as execuções das atividades. Logo, para facilitar a aplicação prática de uma ferramenta de automação, torna-se fundamental que a solução desenvolvida integre com os demais sistemas. Por exemplo, neste relato, os analistas originalmente clonavam os testes no TestLink, mas a tarefa gerava muitos erros pela falta de opções para os dados de teste e problemas de interface. Neste sentido, utilizar a ferramenta para as atividades de especificação e, em seguida, enviar os testes para o TestLink ajudou a diminuir os erros.

7 CONCLUSÃO

O presente artigo apresentou um relato de experiência em uma Fábrica de Testes sobre a geração de procedimentos e casos de testes a partir de casos de uso. O trabalho objetivou analisar a viabilidade de inserção de uma ferramenta semi-automatizada que realizasse essas atividades, antes executadas manualmente. A solução foi implementada a partir de uma colaboração com a indústria, permitindo a geração de um produto que melhor se adequasse às necessidades da equipe de requisitos/testes. Os resultados do uso da ferramenta deram indícios de que a mesma contribuiu com as atividades dos analistas. Também foi observado que a automação está ligada ao seguimento do padrão de escrita, além de ter um ganho pequeno para casos de uso de baixa complexidade. Todavia, o uso da ferramenta trouxe ganhos para casos de uso de maior complexidade. Além disso, foi observado que só o caso de uso não apresentava todas as informações necessárias para uma boa cobertura, sendo necessário automatizar a extração de outras informações.

Adicionalmente aos dados de esforço apresentados, as lições aprendidas discutidas neste artigo podem servir de insumo para projetos similares na indústria e que necessitam automatizar a geração de procedimentos de testes. Para melhor do nosso conhecimento, poucos trabalhos recentes atuais têm reportado informações de esforço do uso das soluções em Fábricas de Testes. Em conjunto com os resultados apresentados nesses outros trabalhos, essas informações podem auxiliar sobre o tipo de solução que melhor se ajusta às necessidades do projeto.

Como trabalhos futuros, foi planejado pesquisar como documentar as regras de negócio, protótipos de telas e domínio dos dados para aumentar a eficiência de geração de procedimentos e casos de teste. Adicionalmente, a solução desenvolvida será aplicada em outros projetos de software e em mais *sprints* do Software A. Por fim, pretende-se analisar como os procedimentos de testes gerados pela ferramenta podem auxiliar na geração semi-automática de scripts de automação de testes funcionais.

REFERÊNCIAS

- [1] Saswat Anand, Edmund K Burke, Tsong Yueh Chen, John Clark, Myra B Cohen, Wolfgang Grieskamp, Mark Harman, Mary Jean Harrold, Phil McMinn, Antonia Bertolino, et al. 2013. An orchestrated survey of methodologies for automated software test case generation. *Journal of Systems and Software* 86, 8 (2013), 1978–2001.
- [2] Bruno Aragão, Ismayle Santos, Tales Nogueira, Lana Mesquita, and Rossana Andrade. 2017. Modelagem Iterativa de um Processo de Desenvolvimento com Base na Percepção da Equipe: Um Relato de Experiência. In *Anais do XIII Simpósio Brasileiro de Sistemas de Informação*. SBC, 428–435.
- [3] Antonia Bertolino. 2007. Software Testing Research: Achievements, Challenges, Dreams. In *2007 Future of Software Engineering (FOSE '07)*. IEEE Computer Society, Washington, DC, USA, 85–103. <https://doi.org/10.1109/FOSE.2007.25>
- [4] Rossana M. de Castro Andrade, Ismayle de Sousa Santos, Valéria Lelli, Kátia Marçal de Oliveira, and Ana Regina Cavalcanti da Rocha. 2017. Software Testing Process in a Test Factory - From Ad hoc Activities to an Organizational Standard. In *ICEIS*.
- [5] Frank Elberzhager, Alla Rosbach, Jürgen Münch, and Robert Eschbach. 2012. Reducing test effort: A systematic mapping study on existing approaches. *Information and Software Technology* 54, 10 (2012), 1092–1106.
- [6] Dietmar Freudenstein, Maximilian Junker, Jeannette Radduenz, Sebastian Eder, and Benedikt Hauptmann. 2018. Automated Test-Design from Requirements-the Specmate Tool. In *2018 IEEE/ACM 5th International Workshop on Requirements Engineering and Testing (RET)*. IEEE, 5–8.
- [7] Vahid Garousi and Michael Felderer. 2017. Living in two different worlds: A comparison of industry and academic focus areas in software testing. *IEEE Software* 1 (2017), 1–1.
- [8] Vahid Garousi and Mika V Mäntylä. 2016. When and what to automate in software testing? A multi-vocal literature review. *Information and Software Technology* 76 (2016), 92–117.
- [9] Tony Gorschek, Per Garre, Stig Larsson, and Claes Wohlin. 2006. A model for technology transfer in practice. *IEEE software* 23, 6 (2006), 88–95.
- [10] JJ Gutiérrez, MJ Escalona, and M Mejias. 2015. A model-driven approach for functional test case generation. *Journal of Systems and Software* 109 (2015), 214–228.
- [11] ISO/IEC29119-2. 2013. ISO/IEC/IEEE International Standard - Software and systems engineering –Software testing –Part 2:Test processes. *ISO/IEC/IEEE 29119-2:2013(E)* (Sept 2013), 1–68. <https://doi.org/10.1109/IEEESTD.2013.6588543>
- [12] Dalton N Jorge, Patricia DL Machado, Everton LG Alves, and Wilkerson L Andrade. 2018. Integrating Requirements Specification and Model-Based Testing in Agile Development. In *2018 IEEE 26th International Requirements Engineering Conference (RE)*. IEEE, 336–346.
- [13] Ljubomir Lazic and Nikos Mastorakis. 2008. Cost effective software test metrics. *WSEAS Transactions on Computers* 7, 6 (2008), 599–619.
- [14] Jobson L Massollar, Rafael M de Mello, and Guilherme H Travassos. 2012. Structuring and Verifying Requirement Specifications through Activity Diagrams to Support the Semi-automated Generation of Functional Test Procedures. In *2012 Eighth International Conference on the Quality of Information and Communications Technology*. IEEE, 239–244.
- [15] Anne Mette and Jonassen Hass. 2008. Testing processes. In *Software Testing Verification and Validation Workshop, 2008. ICSTW'08. IEEE International Conference on*. IEEE, 321–327.
- [16] Mariano Angel Montoni, Ana Regina Rocha, and Kival Chaves Weber. 2009. MPS. BR: a successful program for software process improvement in Brazil. *Software Process: Improvement and Practice* 14, 5 (2009), 289–300.
- [17] Glenford J Myers, Tom Badgett, Todd M Thomas, and Corey Sandler. 2004. *The art of software testing*. Vol. 2. Wiley Online Library.
- [18] Sidney Nogueira, Hugo Araujo, Renata Araujo, Juliano Iyoda, and Augusto Sampaio. 2019. Test case generation, selection and coverage from natural language. *Science of Computer Programming* (2019), 84–110.
- [19] Ganesh Samarthyam, Mahesh Muralidharan, and Raghu Kalyan Anna. 2017. Understanding Test Debt. In *Trends in Software Testing*. Springer, 1–17.
- [20] Ana Sanz, Javier Garcia, Javier Saldana, and Antonio Amescua. 2009. A proposal of a process model to create a Test Factory. In *2009 ICSE Workshop on Software Quality*. IEEE, 65–70.
- [21] Ken Schwaber and Mike Beedle. 2002. *Agile software development with Scrum*. Vol. 1. Prentice Hall Upper Saddle River.
- [22] Swati Seela and Ryan Yackel. [n.d.]. *64 Essential Testing Metrics for Measuring Quality Assurance Success*. Retrieved July 08, 2019 from <https://www.qasymphony.com/blog/64-test-metrics/>
- [23] Harry M Sneed. 2018. Requirement-based testing-extracting logical test cases from requirement documents. In *International Conference on Software Quality*. Springer, 60–79.
- [24] Stephane S Some and Xu Cheng. 2008. An approach for supporting system-level test scenarios generation from textual use cases. In *Proceedings of the 2008 ACM symposium on Applied computing*. ACM, 724–729.
- [25] Lucas Sales Vieira, Cayk G Lima Barreto, Erick Barros dos Santos, Bruno Sabóia Aragão, Ismayle de Sousa Santos, and Rossana M Castro Andrade. 2018. Automação de Testes em uma Fábrica de Testes: Um Relato de Experiência. In *Anais do XIV Simpósio Brasileiro de Sistemas de Informação*. SBC, 80–73.
- [26] Karl Wiegiers and Joy Beatty. 2013. *Software requirements*. Pearson Education.
- [27] Tao Yue, Shaikat Ali, and Man Zhang. 2015. RTCM: a natural language based, automated, and practical test case generation framework. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. ACM, 397–408.